

JANATICS DATA ENGINE

System Design · Service Implementation · AI Agent Prompts
Full-Stack Reference for .NET 8 Class Library + React 18 + TypeScript

Document Type	System Design + Development Reference + AI Agent Prompt Pack
Version	1.0.0 · April 2026
Classification	Internal Engineering — Confidential
Stack	.NET 8 · Dapper · Oracle / SQL Server / SQLite · React 18 · TypeScript · Ant Design 5
AI Agents	Cursor · GitHub Copilot · ChatGPT Codex · Antigravity
Organisation	Janatics India Pvt. Ltd.

PART I — SYSTEM DESIGN

1. Executive Summary

Janatics DataEngine is a **provider-agnostic, registry-driven data access class library** built on .NET 8. It exposes two unified HTTP endpoints — `/fetch` and `/forge` — to any consuming .NET application, eliminating per-entity CRUD boilerplate across all Janatics ERP portals.

Core Problems Solved

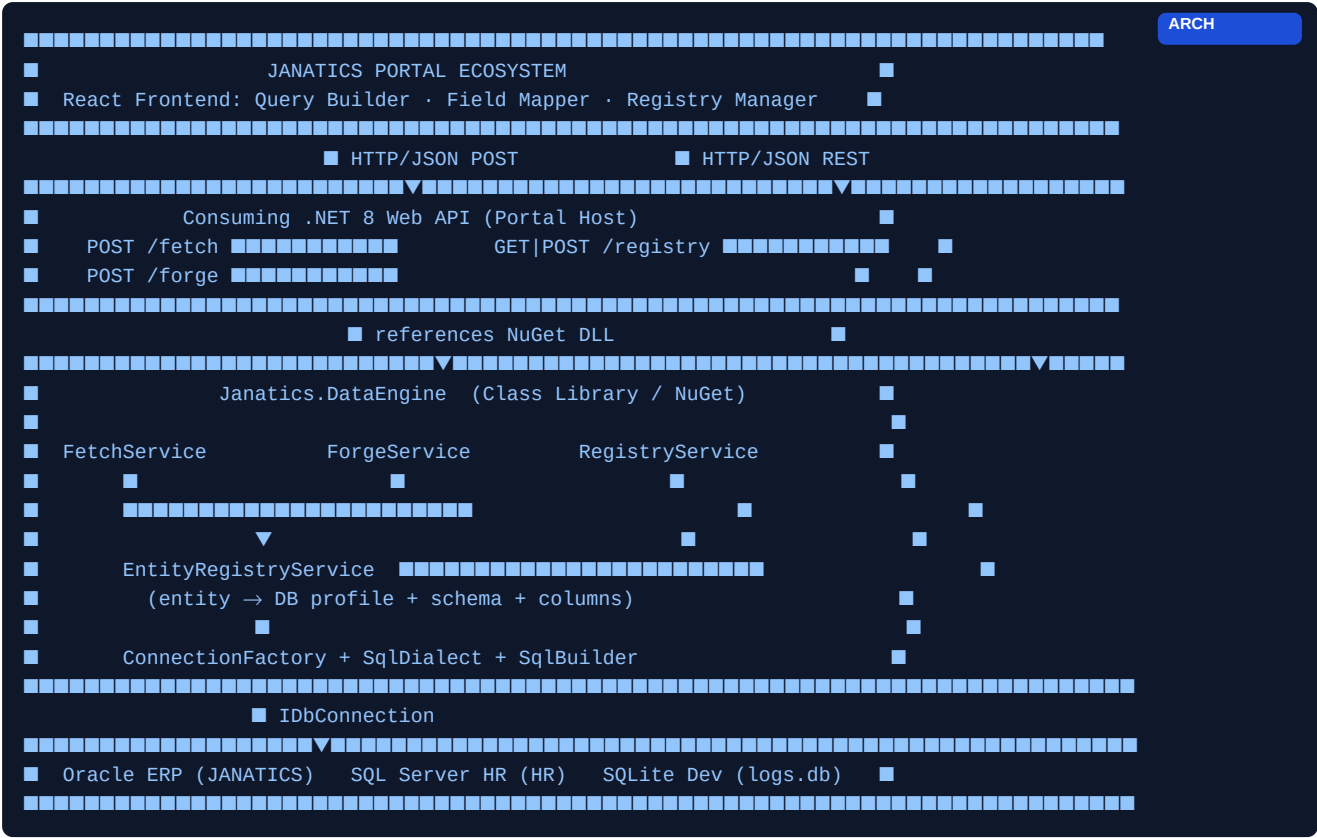
- Every new ERP table required a new Repository + Service + Controller + DTO — 4 files per entity
- Multi-DB routing (Oracle ERP, SQL Server HR, SQLite Dev) was manual and fragmented across projects
- No centralised governance of which tables exist, their PKs, or who is authorised to write to them
- SQL dialect differences (Oracle vs SQL Server vs SQLite) were reimplemented ad hoc in every portal
- Frontend developers had no visual tooling — all payload construction was done by hand in JSON

Value Delivered

Dimension	Before DataEngine	After DataEngine
New entity onboarding	4 files + PR + deploy	1 DB row in Registry UI
DB provider migration	Rewrite query layer	Change config key only
Dev time per CRUD module	2–3 days	< 1 hour
SQL injection surface	Per-developer risk	Registry whitelist enforced
Pagination consistency	Re-impl per endpoint	Unified FetchRequest

2. Architecture Overview

2.1 High-Level System Map



2.2 Architectural Principles

Principle	Implementation
Single Responsibility	SqlBuilder only builds SQL; ConnectionFactory only creates connections; RegistryService only resolves metadata
Open/Closed	New DB providers added by implementing IConnectionFactory — FetchService never changes
Registry Pattern	EntityRegistry is the single source of truth for all table metadata — no hardcoded schema
Fail Fast	WhitelistValidator rejects unknown entities at request boundary before any SQL is constructed
Immutable Payloads	FetchRequest and ForgeRequest are sealed records
Observability First	Every execution emits Serilog structured events with entity, profile, duration, rowCount

3. Registry Data Model (MetaDB)

The Registry is stored in a dedicated lightweight MetaDB (SQL Server or SQLite) independent of any business database.

3.1 DbProfiles Table

Column	Type	Nullable	Description
ProfileId	UNIQUEIDENTIFIER	N	Primary key GUID
ProfileName	NVARCHAR(100)	N	e.g. OracleERP, SqlServerHR
Provider	NVARCHAR(20)	N	Oracle SqlServer Sqlite
Host	NVARCHAR(255)	Y	Server hostname or IP address
Port	INT	Y	Default: Oracle 1521, SQL Server 1433
Database	NVARCHAR(255)	N	DB name or SQLite file path
Username	NVARCHAR(100)	Y	Login username
PasswordHash	NVARCHAR(500)	Y	AES-256 encrypted — never plaintext
Status	NVARCHAR(20)	N	active inactive error

3.2 EntityRegistry Table

Column	Type	Description
EntityId	UNIQUEIDENTIFIER	Primary key GUID
ProfileId	UNIQUEIDENTIFIER	FK → DbProfiles.ProfileId
EntityName	NVARCHAR(200)	Exact DB table name (e.g. PurchaseOrder)
SchemaName	NVARCHAR(100)	Schema prefix (e.g. JANATICS, HR, dbo)
PkColumn	NVARCHAR(100)	Primary key column name (e.g. Id)
IsReadOnly	BIT	Block all forge operations if true
AllowedRoles	NVARCHAR(500)	Comma-separated role names for access control
SoftDeleteColumn	NVARCHAR(100)	Column to set on delete (e.g. IsDeleted)
SoftDeleteValue	NVARCHAR(50)	Value to assign on soft delete (e.g. 1)

3.3 ColumnRegistry Table

Column	Type	Description
ColumnId	UNIQUEIDENTIFIER	Primary key GUID
EntityId	UNIQUEIDENTIFIER	FK → EntityRegistry.EntityId
ColumnName	NVARCHAR(200)	Exact column name in database

Column	Type	Description
DataType	NVARCHAR(50)	string number decimal date boolean guid
IsRequired	BIT	Validated on forge create operations
IsPrimaryKey	BIT	True if this is the PK column
IsForeignKey	BIT	True if this references another entity
FkReference	NVARCHAR(300)	e.g. Vendor.VendorId

4. Janatics.DataEngine — Library Structure

Janatics.DataEngine/ ← .NET 8 Class Library

■■■■ Models/

- ■■■■ FetchRequest.cs ← /fetch payload (page, filters, sort, select, nodes)
- ■■■■ FetchResult.cs ← Paginated response wrapper
- ■■■■ ForgeRequest.cs ← /forge payload (operation, content, nodes)
- ■■■■ ForgeResult.cs ← CUD response with new/affected EntityId
- ■■■■ FilterCondition.cs ← column + op + value (+ value2 for between)
- ■■■■ SearchOptions.cs ← term + columns[]
- ■■■■ SortOptions.cs ← column + direction
- ■■■■ NodeForge.cs / NodeRequest.cs

■■■■ Registry/

- ■■■■ EntityRegistryService.cs ← Queries MetaDB, maps entity → DbProfile + columns
- ■■■■ EntityRegistryCache.cs ← IMemoryCache wrapper, configurable TTL

■■■■ Infrastructure/

- ■■■■ DbConnectionFactory.cs ← Resolves provider, builds IDbConnection
- ■■■■ SqlDialect.cs ← Param prefix (:/@), pagination clause, quoting
- ■■■■ ProviderDetector.cs ← Auto-sniff provider from connection string

■■■■ Builders/

- ■■■■ FetchSqlBuilder.cs ← SELECT + WHERE + SEARCH + JOIN + SORT + PAGE
- ■■■■ ForgeSqlBuilder.cs ← INSERT / UPDATE / soft-DELETE
- ■■■■ WhitelistValidator.cs ← Entity + column name registry validation

■■■■ Services/

- ■■■■ FetchService.cs ← Orchestrates /fetch end-to-end
- ■■■■ ForgeService.cs ← Orchestrates /forge with transaction

■■■■ Exceptions/

- ■■■■ EntityNotFoundException.cs ← 404
- ■■■■ ReadOnlyViolationException.cs ← 403
- ■■■■ RoleViolationException.cs ← 403
- ■■■■ SqlBuildException.cs ← 400

■■■■ Extensions/

- ■■■■ ServiceCollectionExtensions.cs ← AddDataEngine(services, config)

STRUCTURE

Consumer Integration (3 lines)

// Program.cs – consuming portal

```
builder.Services.AddDataEngine(builder.Configuration);
```

// appsettings.json

```
"DataEngine": {
  "MetaDb": { "Provider": "SqlServer", "ConnectionString": "Server=metadb;..." },
  "CacheTtlMinutes": 5,
  "EnableAuditLog": true
}
```

// DataController.cs

```
[HttpPost("/fetch")]
public Task<IActionResult> Fetch([FromBody] FetchRequest req)
    => _fetchService.ExecuteAsync(req, User);

[HttpPost("/forge")]
public Task<IActionResult> Forge([FromBody] ForgeRequest req)
    => _forgeService.ExecuteAsync(req, User);
```

C#

5. API Contracts

5.1 POST /fetch — Read Operation

Field	Type	Default	Description
rootEntity	string	required	Exact entity name from registry
page	int	1	Page number (1-based)
pageSize	int	25	Rows per page — max 1000
includeCount	bool	true	Run COUNT(*) for totalCount
select	string[]	all	Column whitelist — omit for all
filters	array	[]	FilterCondition objects (AND-chained)
search	object	null	term + columns[] for full-text LIKE
sort	object	null	column + direction (asc desc)
nodes	array	[]	Child entities to LEFT JOIN

Filter Operators

Operator	SQL	Types
eq	= :p0	all
neq	!= :p0	all
contains	LIKE '%:p0%'	string
startsWith	LIKE ':p0%'	string
gt / gte	> :p0 / >= :p0	number, decimal, date
lt / lte	< :p0 / <= :p0	number, decimal, date
between	BETWEEN :p0 AND :p1	number, decimal, date
in	IN (:p0,:p1,...)	string, number
isNull	IS NULL	all
isNotNull	IS NOT NULL	all

5.2 POST /forge — Write Operation

Field	Type	Description
rootEntity	string	Entity name from registry
rootId	string?	null = CREATE value = UPDATE or DELETE
operation	string	"create" "update" "delete" (soft)
content	object	Column → value map for root entity
nodes	NodeForge[]	Child records to INSERT or UPDATE in same transaction

/forge Full Example — Create with Child

```
{
  "rootEntity": "PurchaseOrder",
  "rootId": null,
  "operation": "create",
  "content": {
    "PONumber": "PO-2026-001",
    "VendorId": 42,
    "Status": "Draft",
    "OrderDate": "2026-04-28"
  },
  "nodes": [
    {
      "nodeEntity": "PurchaseOrderLine",
      "parentLink": "PurchaseOrderId",
      "nodeId": null,
      "content": { "ItemCode": "JAN-001", "Qty": 10, "UnitPrice": 250.00 }
    }
  ]
}
```

JSON

6. Runtime Execution Flow

6.1 /fetch Pipeline

HTTP POST /fetch → FetchService.ExecuteAsync(req, user)

FLOW

```
[1] EntityRegistryService.LookupAsync("PurchaseOrder")
    ■■■ IMemoryCache HIT → return EntityMeta (zero DB calls)
    ■■■ MISS → SELECT from MetaDB EntityRegistry + DbProfiles
        → cache for CacheTtlMinutes (default 5 min)

[2] WhitelistValidator.Validate(req, entityMeta)
    ■■■ Entity registered? → EntityNotFoundException (404)
    ■■■ Columns in registry? → SqlBuildException (400)
    ■■■ User role in AllowedRoles? → RoleViolationException (403)

[3] ConnectionFactory.Create(profileId)
    ■■■ Decrypt PasswordHash → OracleConnection | SqlConnection | SqliteConnection

[4] FetchSqlBuilder.Build(req, entityMeta, dialect)
    ■■■ SELECT clause (projected or *)
    ■■■ WHERE clause (AND-chained parameterised predicates)
    ■■■ SEARCH clause (LOWER() LIKE across specified columns)
    ■■■ JOIN clause (LEFT JOIN per node)
    ■■■ ORDER BY clause
    ■■■ Pagination:
        Oracle: OFFSET :skip ROWS FETCH NEXT :take ROWS ONLY
        SqlServer: OFFSET @skip ROWS FETCH NEXT @take ROWS ONLY
        SQLite: LIMIT :take OFFSET :skip

[5] Dapper.QueryAsync(sql, parameters, connection)
    + optional ExecuteScalarAsync for COUNT(*)

[6] Return FetchResult { success, data, totalCount, page, totalPages }
    + Serilog: { entity, profile, rows, executionMs, page }
```

6.2 /forge Pipeline

HTTP POST /forge → ForgeService.ExecuteAsync(req, user)

FLOW

```
[1] EntityRegistryService.LookupAsync + WhitelistValidator.ValidateForge
    ■■■ IsReadOnly? → ReadOnlyViolationException (403)
    ■■■ Required columns present for CREATE?
    ■■■ All content keys in ColumnRegistry?

[2] ConnectionFactory.Create(profileId) → Open + BeginTransaction()

[3] ForgeSqlBuilder.BuildParent(req, entityMeta, dialect)
    ■■■ create: INSERT INTO JANATICS.PurchaseOrder (...) VALUES (...)
    ■■■ + dialect returning clause → capture new PK
    ■■■ update: UPDATE JANATICS.PurchaseOrder SET ... WHERE Id = :rootId
    ■■■ delete: UPDATE JANATICS.PurchaseOrder SET IsDeleted = 1 WHERE Id = :rootId

[4] Dapper.ExecuteAsync(parentSql, tx)

[5] For each node: inject parentLink FK = newParentId
    ForgeSqlBuilder.BuildNode → Dapper.ExecuteAsync(nodeSql, tx)

[6] tx.Commit() | tx.Rollback() on any exception

[7] Return ForgeResult { success, entityId, operation, affectedNodes }
```

7. Security Design

Threat	Risk	Mitigation
SQL Injection via entity/column name	HIGH	Registry whitelist validation before SQL construction — unknown names → 400
SQL Injection via filter value	HIGH	All values passed as Dapper parameters — never concatenated
Unauthorised table access	HIGH	AllowedRoles on EntityRegistry enforced against ClaimsPrincipal
Write to read-only entity	MEDIUM	IsReadOnly flag checked in WhitelistValidator for all forge operations
Connection string exposure	HIGH	AES-256 encrypted in MetaDB; decrypted only at runtime; never logged
Mass data extraction	MEDIUM	pageSize capped at 1000; no unpaaginated full-table export path
Forge with unregistered columns	MEDIUM	ColumnRegistry whitelist rejects unknown content keys → 400

Security Note:

Table and column names are interpolated into SQL only after passing WhitelistValidator against the registry. Parameter values are always Dapper-parameterised. AES-256 key stored in DATA_ENGINE_CRYPT0_KEY environment variable only.

8. Frontend Admin Suite

Page	Role	Writes To
Registry Manager	Admin governance — profiles, entities, columns, audit log	MetaDB (EntityRegistry, DbProfiles, ColumnRegistry)
Query Builder	Visual /fetch query composer — filters, sort, pagination	/fetch API (read-only)
Field Mapper	Visual /forge payload builder — INSERT/UPDATE/child nodes	/forge API (write)

Registry Manager Features

- Add / Edit / Delete DB profiles with auto-generated connection string preview and provider badge
- Test Connection button — pings the real DB and updates status in registry
- Auto-Discover tables — queries INFORMATION_SCHEMA / ALL_TABLES and lists unregistered tables for one-click import
- Expandable entity cards with column registry panel showing type badges, PK/FK annotations
- Audit log viewer — filter by entity, action type, changed-by, date range

Query Builder Features

- Entity selector driven by registry — only registered entities shown
- Column selector with type chips and all/none shortcuts; progress bar
- Multi-condition filter builder with operator dropdown scoped by column data type (13 operators)
- Full-text search with per-column scope; sort config with ASC/DESC toggle
- Child join (node) configuration with FK column override
- Pagination with preset page-size buttons (10 / 25 / 50 / 100 / 500)
- Live JSON payload preview + copy-to-clipboard; Execute against real API with inline results table

Field Mapper Features

- INSERT / UPDATE mode toggle — UPDATE mode reveals Transaction ID input
- Column rows auto-generated from ColumnRegistry — required fields marked, data types respected
- Child entity sections with FK column config and Record ID for per-child update mode
- Payload preview tab with validation summary and one-click copy

PART II — AI AGENT PROMPTS FOR CODE GENERATION

The following prompts are optimised for each AI coding agent. Each prompt is self-contained — it carries full context so the agent can scaffold the correct code without requiring prior knowledge of this document. Copy the prompt verbatim into the agent's input.

Agent	Best For	Prompt Style
Cursor	Full-project scaffolding in existing codebase; file-tree aware	Iterative + file paths + architecture rules
GitHub Copilot	File-by-file implementation; inline suggestions inside VS Code	Context + interface contracts first
ChatGPT Codex	Complete working code blocks; boilerplate generation; Sequence generation	Expected output format
Antigravity	Multi-agent orchestration; parallel service + UI generation	Layered task decomposition + dependency order

Cursor

AI AGENT PROMPT · JANATICS DATAENGINE

PROMPT A — Backend Class Library (Janatics.DataEngine)

CURSOR PROMPT

You are scaffolding Janatics.DataEngine, a .NET 8 Class Library (NuGet package).
Generate the complete project with the following exact file structure:

PROJECT CONTEXT

Name: Janatics.DataEngine
Target: net8.0
NuGet deps: Dapper, Oracle.ManagedDataAccess.Core, Microsoft.Data.SqlClient,
Microsoft.Data.Sqlite, Microsoft.Extensions.Caching.Memory,
Serilog, Serilog.Sinks.Console

MODELS TO GENERATE (Models/)

FetchRequest.cs – sealed record with:
 string RootEntity, int Page=1, int PageSize=25, bool IncludeCount=true,
 List<string>? Select, List<FilterCondition> Filters, SearchOptions? Search,
 SortOptions? Sort, List<NodeRequest>? Nodes
FilterCondition.cs – record: string Column, string Op, object? Value, object? Value2
SearchOptions.cs – record: string Term, List<string> Columns
SortOptions.cs – record: string Column, string Direction
NodeRequest.cs – record: string NodeEntity, string ParentLink
FetchResult.cs – record: bool Success, IReadOnlyList<dynamic>? Data,
 int? TotalCount, int Page, int PageSize, string? Error
ForgeRequest.cs – sealed record with:
 string RootEntity, string? RootId, string Operation,
 Dictionary<string,object?> Content, List<NodeForge>? Nodes
NodeForge.cs – record: string NodeEntity, string ParentLink,
 string? NodeId, Dictionary<string,object?> Content
ForgeResult.cs – record: bool Success, string? EntityId, string Operation,
 int AffectedNodes, string? Error

INFRASTRUCTURE (Infrastructure/)

SqlDialect.cs – value type with:
 string ParamPrefix (: for Oracle/Sqlite, @ for SqlServer)
 string PaginationClause(int skip, int take)
 string QuoteIdentifier(string name)
 string NewIdClause(string tableName) // SCOPE_IDENTITY / last_insert_rowid
 Param(string col) => ParamPrefix + col

DbConnectionFactory.cs – singleton, implements IDbConnectionFactory:
 Create(string profileId) => resolve from internal Dictionary<string, DbProfile>
 Supports OracleConnection, SqlConnection, SqliteConnection

Decrypts PasswordHash with AES-256-CBC using env var DATA_ENGINE_CRYPTO_KEY

REGISTRY (Registry/)

EntityRegistryService.cs – scoped service:
 Task<EntityMeta> LookupAsync(string entityName)
 - Check IMemoryCache key "entity:{entityName}"
 - On miss: SELECT from MetaDB EntityRegistry JOIN DbProfiles
 then SELECT ColumnRegistry for entity
 - Cache with absolute expiration from config CacheTtlMinutes
 - Throw EntityNotFoundException if not found

BUILDERS (Builders/)

WhitelistValidator.cs:
 ValidateFetch(FetchRequest, EntityMeta, ColumnMeta[]) – throws typed exceptions

ValidateForge(ForgeRequest, EntityMeta, ColumnMeta[]) – checks IsReadOnly, roles, columns

FetchSqlBuilder.cs – static pure function Build(FetchRequest, EntityMeta, SqlDialect):
Returns (string Sql, Dictionary<string,object?> Parameters)
Build SELECT, WHERE (AND-chained parameterised), SEARCH (OR LIKE),
LEFT JOIN per node, ORDER BY, pagination clause

ForgeSqlBuilder.cs – static pure function Build(ForgeRequest, EntityMeta, SqlDialect):
Returns (string Sql, Dictionary<string,object?> Parameters)
create: INSERT + dialect NewIdClause
update: UPDATE SET ... WHERE pkColumn = param
delete: UPDATE SET softDeleteColumn = softDeleteValue WHERE pkColumn = param

SERVICES (Services/)

FetchService.cs – scoped, implements IFetchService:
ExecuteAsync(FetchRequest, ClaimsPrincipal)
→ Lookup → Validate → Connect → Build → Dapper.QueryAsync
→ optional COUNT → return FetchResult
Emit Serilog Information event on success, Error on exception

ForgeService.cs – scoped, implements IForgeService:
ExecuteAsync(ForgeRequest, ClaimsPrincipal)
→ Lookup → Validate → Connect → BeginTransaction
→ BuildParent → Execute → BuildNode(s) → Execute → Commit
→ Rollback on exception → return ForgeResult

EXTENSIONS (Extensions/)

ServiceCollectionExtensions.cs:
AddDataEngine(IServiceCollection, IConfiguration)
Registers: IEntityRegistryService, IDbConnectionFactory,
IFetchService, IForgeService, IMemoryCache
Reads: config section "DataEngine" for MetaDb + CacheTtlMinutes + EnableAuditLog

EXCEPTIONS (Exceptions/)

DataEngineException (base) : Exception
EntityNotFoundException : DataEngineException → maps to HTTP 404
ReadOnlyViolationException : DataEngineException → HTTP 403
RoleViolationException : DataEngineException → HTTP 403
SqlBuildException : DataEngineException → HTTP 400
ConnectionException : DataEngineException → HTTP 503

RULES:

- Use C# 12 primary constructors and record types where applicable
- No EF Core – Dapper only
- All public APIs have XML doc comments
- Unit-testable: no static dependencies except FetchSqlBuilder/ForgeSqlBuilder
- Generate xUnit test stubs in Janatics.DataEngine.Tests/ for all builder methods

PROMPT B — React Frontend Suite

CURSOR PROMPT

You are scaffolding the React 18 + TypeScript admin frontend for Janatics DataEngine.
The backend exposes POST /fetch and POST /forge and REST endpoints under /api/registry.

TECH STACK

React 18, TypeScript 5, Vite 5
Ant Design 5.x (primary UI component library)
TanStack Query v5 (data fetching + cache)
TanStack Router v1 (file-based routing)
Axios (HTTP client)
Zod (schema validation for all API payloads)

FOLDER STRUCTURE TO GENERATE

```
src/
├── types/
│   ├── FetchRequest.ts      - Zod schema + inferred type
│   ├── ForgeRequest.ts      - Zod schema + inferred type
│   └── Registry.ts          - DbProfile, EntityMeta, ColumnMeta types
├── services/
│   ├── fetchApi.ts          - postFetch(req: FetchRequest): Promise<FetchResult>
│   ├── forgeApi.ts          - postForge(req: ForgeRequest): Promise<ForgeResult>
│   └── registryApi.ts        - CRUD for profiles, entities, columns
├── pages/
│   ├── RegistryManager.tsx   - Full DB registry admin page
│   ├── QueryBuilder.tsx      - Visual /fetch query builder
│   └── FieldMapper.tsx       - Visual /forge payload builder
├── components/
│   ├── registry/
│   │   ├── ProfileCard.tsx
│   │   ├── ProfileForm.tsx
│   │   ├── EntityCard.tsx
│   │   ├── EntityForm.tsx
│   │   └── ColumnPanel.tsx
│   ├── querybuilder/
│   │   ├── FilterRow.tsx
│   │   ├── ColumnSelector.tsx
│   │   ├── SortConfig.tsx
│   │   └── ResultsTable.tsx
│   └── fieldmapper/
│       ├── ColumnRow.tsx
│       └── ChildSection.tsx
```

REGISTRY MANAGER PAGE

- Left sidebar: list of DbProfiles (name, provider badge, status dot, entity count)
- Main panel for selected profile:
 - Profile detail card with Test Connection button (POST /api/registry/profiles/{id}/test)
 - Auto-Discover button (POST /api/registry/profiles/{id}/discover)
 - shows discovered table names with Import buttons
 - Search input filtering entity cards
 - EntityCard per registered entity:
 - expandable to show column list with type/PK/FK badges
 - Edit button → EntityForm modal (Ant Design Modal + Form)
 - Remove button with Popconfirm
 - Columns button → ColumnPanel drawer
- Add Profile button → ProfileForm modal
- Add Entity button (per profile) → EntityForm modal

QUERY BUILDER PAGE

Layout: Left panel (420px) | Centre payload preview (320px) | Right results
Left panel tabs: Columns | Filters | Search | Sort | Joins | Pagination
Columns tab: checkbox list from /api/registry/entities/{id}/columns

Filters tab: dynamic list of FilterRow components
FilterRow: column select → type badge → operator select → value input(s)
Operators filtered by column data type
Search tab: term input + column checkboxes (string/number types only)
Sort tab: column select + ASC/DESC toggle buttons
Joins tab: list of child entities from registry with Include toggle + FK input
Pagination tab: page number stepper, page size preset buttons, includeCount toggle
Centre panel: JSON payload updated on every field change + Copy button
Right panel: Execute button → calls POST /fetch → renders data table
Table: column headers from data keys, prev/next pagination controls
Show totalCount, page, totalPages in header

FIELD MAPPER PAGE

Top bar: entity selector + INSERT/UPDATE toggle
UPDATE mode: show Transaction ID input (maps to rootId)
Column rows: one row per registered column
Each row: columnName label | type badge | → arrow | value input (type-aware)
Required columns marked with *; unfilled required columns highlighted
Progress bar: filled columns / total columns
Child entity section: Add [ChildEntityName] button per registered child

Child section: FK column config + Record ID input + column rows
Two tabs: Mapper view | Payload Preview (JSON + validation summary)
Submit button: POST /forge → show success/error notification

RULES:

- All API calls through TanStack Query (useQuery / useMutation)
- All form state via Ant Design Form with Zod resolver
- No inline styles – use Ant Design tokens + CSS Modules
- Generate TypeScript types from Zod schemas (z.infer<typeof Schema>)
- All components have prop types defined as interfaces
- Loading states and error states handled in every data-fetching component

■ GitHub Copilot

AI AGENT PROMPT · JANATICS DATAENGINE

PROMPT C — .NET 8 Service Implementation

COPILOT PROMPT

```
// COPILOT CONTEXT — Janatics.DataEngine Class Library
// Implement each file below. Open the file, add the comment block,
// then let Copilot complete the implementation.

// FILE: Infrastructure/SqlDialect.cs
// Implement a value type SqlDialect for .NET 8.
// Constructor: SqlDialect(string provider) where provider is "Oracle","SqlServer","Sqlite"
// Properties:
//   string ParamPrefix → ":" for Oracle/Sqlite, "@" for SqlServer
// Methods:
//   string Param(string columnName) → ParamPrefix + columnName
//   string PaginationClause(int skip, int take)
//     Oracle/SqlServer: OFFSET {skip} ROWS FETCH NEXT {take} ROWS ONLY
//     Sqlite: LIMIT {take} OFFSET {skip}
//   string QuoteIdentifier(string name)
//     Oracle: name.ToUpper() | SqlServer: [name] | Sqlite: "name"
//   string NewIdClause()
//     SqlServer: "; SELECT CAST(SCOPE_IDENTITY() AS NVARCHAR(50))"
//     Sqlite: "; SELECT last_insert_rowid()"
//     Oracle: "" (Oracle uses OracleParameter with Direction.Output separately)

// FILE: Builders/FetchSqlBuilder.cs
// Implement static class FetchSqlBuilder.
// Method: Build(FetchRequest req, EntityMeta meta, SqlDialect dialect)
// Returns: (string Sql, Dictionary<string,object?> Parameters)
// Algorithm:
// 1. SELECT: if req.Select null/empty → SELECT * else SELECT col1,col2,...
//    Always prefix with schema: {meta.SchemaName}.{meta.EntityName}
// 2. WHERE: foreach filter in req.Filters (skip if column empty):
//    Build predicate string per filter.Op:
//    eq/neq/gt/gte/lt/lte: "{col} {sqlOp} {dialect.Param("p"+i)}"
//    contains: "LOWER({col}) LIKE {dialect.Param("p"+i)}" value = "{val}%"
//    between: "{col} BETWEEN {Param("p"+i)} AND {Param("p"+i+"b")}"
//    in: "{col} IN ({comma-joined params})" split value on comma
//    isNull: "{col} IS NULL" isNotNull: "{col} IS NOT NULL"
//    Join all predicates with AND
// 3. SEARCH: if req.Search != null && term != empty:
//    foreach col in req.Search.Columns (or all string cols from meta):
//    "LOWER({col}) LIKE {Param("search")}" → add to WHERE with OR
//    Parameter search value = "{term.ToLower()}"

// 4. ORDER BY: if req.Sort != null → ORDER BY {col} {direction}
// 5. WRAP with dialect.PaginationClause((req.Page-1)*req.PageSize, req.PageSize)
// If req.IncludeCount: also build COUNT(*) query with same WHERE (no pagination)

// FILE: Builders/ForgeSqlBuilder.cs
// Implement static class ForgeSqlBuilder.
// Method: Build(ForgeRequest req, EntityMeta meta, SqlDialect dialect)
// Returns: (string Sql, Dictionary<string,object?> Parameters)
// Algorithm:
// operation == "create":
//   cols = req.Content.Keys.ToList()
//   INSERT INTO {schema}.{table} ({cols.Join(",")}) VALUES ({params.Join(",")})
//   + dialect.NewIdClause()
// operation == "update":
```

```
//      setClauses = Content.Keys.Where(k != pkCol).Select(k => "{k} = {Param(k)}")
//      UPDATE {schema}.{table} SET {setClauses.Join(", ")} WHERE {pkCol} = {Param("__id")}
//      Add __id = req.RootId to parameters
//      operation == "delete":
//      UPDATE {schema}.{table} SET {meta.SoftDeleteColumn} = {Param("__sdv")}
//      WHERE {pkCol} = {Param("__id")}
//      Add __sdv = meta.SoftDeleteValue, __id = req.RootId

// FILE: Services/FetchService.cs
// Implement class FetchService : IFetchService
// Constructor: inject IEntityRegistryService, IDbConnectionFactory, ILogger<FetchService>
// Method: async Task<FetchResult> ExecuteAsync(FetchRequest req, ClaimsPrincipal user)
// 1. var meta = await _registry.LookupAsync(req.RootEntity)
// 2. _validator.ValidateFetch(req, meta, user)
// 3. var dialect = new SqlDialect(meta.Provider)
// 4. using var conn = _factory.Create(meta.ProfileId); conn.Open()
// 5. var (sql, p) = FetchSqlBuilder.Build(req, meta, dialect)
// 6. var data = await conn.QueryAsync<dynamic>(sql, p)
// 7. int? count = req.IncludeCount ? await conn.ExecuteScalarAsync<int>(countSql,p) : null
// 8. return new FetchResult(true, data.ToList(), count, req.Page, req.PageSize, null)
// Wrap in try/catch → log error → return FetchResult(false,...,ex.Message)
```

PROMPT D — React Components (Copilot Chat)

COPILOT PROMPT

```
// COPILOT CHAT — React TypeScript Components for DataEngine Frontend
// Use these prompts in GitHub Copilot Chat one file at a time.

// === PROMPT FOR: src/types/FetchRequest.ts ===
// Create a Zod schema called fetchRequestSchema for a DataEngine /fetch payload.
// Fields: rootEntity (string, min 1), page (number, default 1),
// pageSize (number, default 25, max 1000), includeCount (boolean, default true),
// select (string[] optional), sort (object {column, direction "asc"|"desc"} optional),
// search (object {term: string, columns: string[]} optional),
// filters (array of {column: string, op: z.enum([13 operators]), value?, value2?}),
// nodes (array of {nodeEntity: string, parentLink: string} optional)
// Export: fetchRequestSchema, FetchRequest = z.infer<typeof fetchRequestSchema>

// === PROMPT FOR: src/types/ForgeRequest.ts ===
// Create a Zod schema forgeRequestSchema.
// Fields: rootEntity (string), rootId (string nullable),
// operation (z.enum(["create","update","delete"])),
// content (z.record(z.unknown())), nodes (array of NodeForge optional)
// NodeForge: nodeEntity, parentLink, nodeId (nullable), content (z.record)
// Export schema + inferred type

// === PROMPT FOR: src/services/fetchApi.ts ===
// Create an Axios-based API service for /fetch.
// Export: postFetch(req: FetchRequest): Promise<FetchResult>
// FetchResult interface: success, data: Record<string,unknown>[],
// totalCount?: number, page, pageSize, totalPages?: number, error?: string
// Base URL from import.meta.env.VITE_API_BASE_URL

// === PROMPT FOR: src/services/registryApi.ts ===
// Create Axios CRUD service for the DataEngine registry.
// Exports:
//   getProfiles(): Promise<DbProfile[]>
//   createProfile(data): Promise<DbProfile>
//   updateProfile(id, data): Promise<DbProfile>
//   deleteProfile(id): Promise<void>
//   testConnection(id): Promise<{status: string, latencyMs: number}>
//   discoverTables(id): Promise<string[]>
//   getEntities(profileId): Promise<EntityMeta[]>
//   createEntity(profileId, data): Promise<EntityMeta>
//   updateEntity(id, data): Promise<EntityMeta>
//   deleteEntity(id): Promise<void>
//   getColumns(entityId): Promise<ColumnMeta[]>
//   createColumn(entityId, data): Promise<ColumnMeta>
//   deleteColumn(id): Promise<void>

// === PROMPT FOR: src/components/querybuilder/FilterRow.tsx ===
// Create FilterRow React component for DataEngine Query Builder.
// Props: filter (FilterCondition), columns (ColumnMeta[]),
//   onChange (FilterCondition) => void, onRemove () => void
// Render: AND badge | AntD Select for column | type badge |
//   AntD Select for operator (options filtered by column.dataType) |
//   AntD Input/InputNumber/DatePicker based on type |
//   second input for "between" op | remove button
// Operator groups: string→[eq,neq,contains,startsWith,endsWith,in,isNull,isNotNull]
//   number/decimal→[eq,neq,gt,gte,lt,lte,between,in,isNull,isNotNull]
//   date→[eq,neq,gt,gte,lt,lte,between,isNull,isNotNull]
//   boolean→[eq,isNull,isNotNull]
// isNull/isNotNull ops should hide value inputs
```

■ ChatGPT Codex

AI AGENT PROMPT · JANATICS DATAENGINE

PROMPT E — Complete Backend Implementation

CODEX PROMPT E

Generate complete, production-ready C# code for the Janatics.DataEngine .NET 8 class library. This is a dynamic data access engine – it does NOT use EF Core or typed models for business entities. It uses Dapper with runtime-generated SQL.

ARCHITECTURE RULES:

- All SQL is generated dynamically at runtime from JSON payloads
- Table/column names come from a registry (never from user input directly)
- All filter values use Dapper parameterisation (never string concatenation)
- Supports Oracle (param prefix :), SQL Server (param prefix @), SQLite
- Parent + all child node operations execute inside a single IDbTransaction
- Soft delete: UPDATE SET {softDeleteColumn} = {softDeleteValue} – never DELETE FROM

GENERATE THESE COMPLETE FILES WITH FULL IMPLEMENTATION:

1. Models/FetchRequest.cs

```
public sealed record FetchRequest {
    public string RootEntity { get; init; } = "";
    public int Page { get; init; } = 1;
    public int PageSize { get; init; } = 25;
    public bool IncludeCount { get; init; } = true;
    public List<string>? Select { get; init; }
    public List<FilterCondition> Filters { get; init; } = new();
    public SearchOptions? Search { get; init; }
    public SortOptions? Sort { get; init; }
    public List<NodeRequest>? Nodes { get; init; }
}
```

2. Models/ForgeRequest.cs

```
public sealed record ForgeRequest {
    public string RootEntity { get; init; } = "";
    public string? RootId { get; init; }
    public string Operation { get; init; } = "create"; // create|update|delete
    public Dictionary<string, object?> Content { get; init; } = new();
    public List<NodeForge>? Nodes { get; init; }
}
```

3. Infrastructure/SqlDialect.cs – full implementation (see algorithm in Copilot prompt)

4. Builders/FetchSqlBuilder.cs – full implementation with all 13 operators:

eq, neq, contains, startsWith, endsWith, gt, gte, lt, lte,

between, in, isNull, isNotNull

Handles: SELECT projection, WHERE, SEARCH (LOWER LIKE), LEFT JOIN nodes,
ORDER BY, provider-specific pagination, optional COUNT query

5. Builders/ForgeSqlBuilder.cs – full implementation for create/update/delete(soft)

NodeForge child records use same logic with FK injection

6. Builders/WhitelistValidator.cs – validates:

- Entity exists in registry → EntityNotFoundException
- All select/filter columns in ColumnRegistry → SqlBuildException
- IsReadOnly on forge → ReadOnlyViolationException
- AllowedRoles intersection with ClaimsPrincipal roles → RoleViolationException
- Required columns present on create → SqlBuildException

```
7. Services/FetchService.cs – full implementation with Stopwatch + Serilog logging
Log: Information "FetchExecuted {Entity} {Rows} rows in {ElapsedMs}ms"
Log: Error "FetchFailed {Entity} {Error}"

8. Services/ForgeService.cs – full implementation with transaction + Serilog logging
Log: Information "ForgeExecuted {Entity} {Operation} id={EntityId} in {ElapsedMs}ms"
Log: Error "ForgeFailed {Entity} {Operation} – rollback executed: {Error}"

9. Extensions/ServiceCollectionExtensions.cs
public static IServiceCollection AddDataEngine(this IServiceCollection services,
    IConfiguration configuration)
Register: IEntityRegistryService (Scoped), IDbConnectionFactory (Singleton),
    IFetchService (Scoped), IForgeService (Scoped), IMemoryCache
Bind config from "DataEngine" section

OUTPUT FORMAT:
- Each file starts with // FILE: <path>
- Complete using statements
- XML doc comments on all public members
- No placeholder TODO comments – full working code
- Use C# 12 features: primary constructors, collection expressions, pattern matching
```

PROMPT F — .NET API Endpoint Controller

CODEX PROMPT F

Generate a complete ASP.NET Core 8 Minimal API or Controller setup that consumes the Janatics.DataEngine class library. This is the HOST project – the thin layer that exposes the DLL as HTTP endpoints.

GENERATE: Controllers/DataController.cs

```
[ApiController]
[Route("api")]
public class DataController : ControllerBase
{
    // Constructor: inject IFetchService, IForgeService

    // POST /api/fetch
    // Body: FetchRequest (validated with [Required] and model state)
    // Returns: 200 FetchResult | 400 validation error | 403 role/readonly | 404 entity
    // Add [Authorize] attribute – reads roles from JWT claims
    [HttpPost("fetch")]
    public async Task<IActionResult> Fetch([FromBody] FetchRequest req)

    // POST /api/forge
    // Body: ForgeRequest
    // Returns: 200 ForgeResult | 400 | 403 | 404
    [HttpPost("forge")]
    public async Task<IActionResult> Forge([FromBody] ForgeRequest req)
}
```

GENERATE: Controllers/RegistryController.cs

```
[ApiController]
[Route("api/registry")]
[Authorize(Roles = "admin")]
public class RegistryController : ControllerBase
{
    // GET /api/registry/profiles → List<DbProfileDto>
    // POST /api/registry/profiles → DbProfileDto (create)
    // PUT /api/registry/profiles/{id} → DbProfileDto (update)
    // DELETE /api/registry/profiles/{id} → 204
    // POST /api/registry/profiles/{id}/test → { status, latencyMs }
    // POST /api/registry/profiles/{id}/discover → string[] (unregistered tables)
    // POST /api/registry/profiles/{id}/invalidate → 204 (clears cache)
```

```
// GET /api/registry/entities?profileId={id} → List<EntityMetaDto>
// POST /api/registry/entities → EntityMetaDto
// PUT /api/registry/entities/{id} → EntityMetaDto
// DELETE /api/registry/entities/{id} → 204

// GET /api/registry/entities/{id}/columns → List<ColumnMetaDto>
// POST /api/registry/entities/{id}/columns → ColumnMetaDto
// DELETE /api/registry/columns/{id} → 204
}
```

GENERATE: Middleware/DataEngineExceptionMiddleware.cs

Map DataEngineException subtypes to HTTP status codes:

```
EntityNotFoundException → 404
RoleViolationException → 403
ReadOnlyViolationException → 403
SqlBuildException → 400
ConnectionException → 503
```

Return JSON: { success: false, error: message, errorCode: typeName, traceId }

```
GENERATE: Program.cs snippet
builder.Services.AddDataEngine(builder.Configuration)
app.UseMiddleware<DataEngineExceptionMiddleware>()
app.UseAuthentication(); app.UseAuthorization()
app.MapControllers()
Configure Serilog from appsettings.json
```

AI AGENT PROMPT · JANATICS DATAENGINE

ANTIGRAVITY PR

PHASE 1 – Foundation (no dependencies)

PHASE 2 – Infrastructure (depends on Phase 1)

Page 24

TASK 3B – ForgeSqlBuilder (full implementation)
create/update/delete(soft) for parent and child nodes.
Input→Output: (ForgeRequest, EntityMeta, SqlDialect) → (Sql, Parameters)

TASK 3C – WhitelistValidator (full implementation)
ValidateFetch + ValidateForge. All 5 exception types used.

```
TASK 3D – EntityRegistryService
IMemoryCache lookup + MetaDB query via Dapper.
Load EntityMeta + ColumnMeta[] in one logical lookup (2 SQL queries).
Cache with absolute expiration.
```

```
TASK 3E – React API Services
src/services/fetchApi.ts – postFetch with Axios, full FetchResult type
src/services/forgeApi.ts – postForge with Axios, full ForgeResult type
src/services/registryApi.ts – full CRUD + testConnection + discoverTables
```

PHASE 4 – Services + UI Components (depends on Phase 3)

TASK 4A – FetchService.cs

Orchestrate: Lookup → Validate → Dialect → Connect → Build → Query → Return

With Stopwatch timing and Serilog structured logging.

```
TASK 4B - ForgeService.cs
Orchestrate: Lookup → Validate → Connect → Transaction → BuildParent →
            Execute → BuildNodes → Execute → Commit | Rollback
With timing and Serilog logging.
```

```
TASK 4C – QueryBuilder React page
5-section left panel (tabs: Columns, Filters, Search, Sort, Pagination, Joins)
FilterRow component for each filter with type-aware operator dropdown
Live JSON payload preview | Execute → results table with pagination
```

```
TASK 4D - FieldMapper React page
  INSERT/UPDATE toggle + entity selector
  Per-column rows from ColumnMeta | child entity sections
  Payload preview tab | Submit → POST /forge
```

TASK 4E – RegistryManager React page

Profile sidebar	Entity cards with expand	Column panel drawer
Test Connection	Auto-Discover + Import	Add/Edit/Delete modals

PHASE 5 – Integration + API Layer (depends on Phase 4)

```
TASK 5A – DataController + RegistryController
Full ASP.NET Core 8 controllers calling IFetchService + IForgeService
DataEngineExceptionMiddleware mapping exceptions to HTTP status codes
```

TASK 5B – ServiceCollectionExtensions

AddDataEngine() wiring all services + MetaDB connection + cache

```
TASK 5C – xUnit Test Project
FetchSqlBuilderTests – all 13 operators, pagination variants per dialect
ForgeSqlBuilderTests – create/update/delete across all providers
WhitelistValidatorTests – all violation scenarios
FetchServiceTests – mock IEntityRegistryService + IDbConnectionFactory
```

COMPLETION CRITERIA:

```
dotnet build → 0 errors  
dotnet test → all green  
npm run type-check → 0 TypeScript errors
```

```
POST /fetch with { rootEntity: "PurchaseOrder", pageSize: 10 } → HTTP 200  
POST /forge with create payload → HTTP 200 with entityId returned
```

PROMPT H — MetaDB SQL Migration Script

SQL MIGRATION

Generate a complete SQL migration script for the DataEngine MetaDB.
Must be compatible with both SQL Server 2019+ and SQLite 3.38+.
Use conditional syntax where dialects differ.

```
-- SQL Server version:
```

```
CREATE TABLE DbProfiles (
  ProfileId      UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID() PRIMARY KEY,
  ProfileName    NVARCHAR(100)    NOT NULL UNIQUE,
  Provider       NVARCHAR(20)     NOT NULL CHECK(Provider IN ('Oracle','SqlServer','Sqlite')),
  Host          NVARCHAR(255)     NULL,
  Port           INT              NULL,
  [Database]     NVARCHAR(255)    NOT NULL,
  Username       NVARCHAR(100)    NULL,
  PasswordHash   NVARCHAR(500)    NULL,
  Status         NVARCHAR(20)     NOT NULL DEFAULT 'active',
  CreatedAt      DATETIME2        NOT NULL DEFAULT GETUTCDATE(),
  UpdatedAt      DATETIME2        NOT NULL DEFAULT GETUTCDATE()
);

CREATE TABLE EntityRegistry (
  EntityId      UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID() PRIMARY KEY,
  ProfileId      UNIQUEIDENTIFIER NOT NULL REFERENCES DbProfiles(ProfileId),
  EntityName     NVARCHAR(200)    NOT NULL,
  SchemaName     NVARCHAR(100)    NOT NULL DEFAULT 'dbo',
  PkColumn       NVARCHAR(100)    NOT NULL DEFAULT 'Id',
  IsReadOnly     BIT              NOT NULL DEFAULT 0,
  AllowedRoles   NVARCHAR(500)    NULL,
  SoftDeleteColumn NVARCHAR(100)  NULL,
  SoftDeleteValue NVARCHAR(50)    NULL DEFAULT '1',
  CreatedAt      DATETIME2        NOT NULL DEFAULT GETUTCDATE(),
  CreatedBy      NVARCHAR(100)    NOT NULL DEFAULT SYSTEM_USER,
  CONSTRAINT UQ_Entity_Profile UNIQUE (ProfileId, EntityName)
);

CREATE TABLE ColumnRegistry (
  ColumnId      UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID() PRIMARY KEY,
  EntityId      UNIQUEIDENTIFIER NOT NULL REFERENCES EntityRegistry(EntityId) ON DELETE CASCADE,
  ColumnName     NVARCHAR(200)    NOT NULL,
  DataType       NVARCHAR(50)     NOT NULL DEFAULT 'string',

  IsRequired     BIT              NOT NULL DEFAULT 0,
  IsPrimaryKey   BIT              NOT NULL DEFAULT 0,
  IsForeignKey   BIT              NOT NULL DEFAULT 0,
  FkReference     NVARCHAR(300)    NULL,
  CreatedAt      DATETIME2        NOT NULL DEFAULT GETUTCDATE(),
  CONSTRAINT UQ_Column_Entity UNIQUE (EntityId, ColumnName)
);

CREATE TABLE AuditLog (
  LogId         UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID() PRIMARY KEY,
  ProfileId     UNIQUEIDENTIFIER NULL REFERENCES DbProfiles(ProfileId),
  EntityId     UNIQUEIDENTIFIER NULL REFERENCES EntityRegistry(EntityId),
  Action       NVARCHAR(100)     NOT NULL,
  ChangedBy    NVARCHAR(200)     NOT NULL,
  ChangedAt    DATETIME2        NOT NULL DEFAULT GETUTCDATE(),
  OldValue     NVARCHAR(MAX)     NULL,
  NewValue     NVARCHAR(MAX)     NULL
);

-- Seed: default OracleERP profile (PasswordHash to be updated)
```

```
INSERT INTO DbProfiles (ProfileName, Provider, Host, Port, [Database], Username)
VALUES ('OracleERP', 'Oracle', 'erp.janatics.local', 1521, 'ERPDB', 'erp_user');

-- Also generate the equivalent SQLite DDL with TEXT/INTEGER types.
```

PART III — QUICK REFERENCE

Prompt Usage Matrix

Prompt	Agent	Generates	Use When
A	Cursor	Full DLL file tree + all classes	Starting the backend from scratch in existing repo
B	Cursor	Full React src/ folder structure	Starting the frontend from scratch
C	Copilot	Individual .cs file implementations	Implementing file-by-file in VS Code
D	Copilot	Individual .tsx component files	Building React components one at a time
E	Codex	Complete C# code blocks with full logic	Need complete working code for all classes
F	Codex	ASP.NET controller + middleware + Program.cs	Setting up the API host project
G	Antigravity	Phased multi-agent task decomposition	Parallel agent orchestration / team workflow
H	Any agent	SQL Server + SQLite MetaDB migration script	Creating the MetaDB schema

Payload Quick Reference

/fetch — Minimal

```
{ "rootEntity": "PurchaseOrder", "page": 1, "pageSize": 25 }
```

JSON

/fetch — Full

```
{
  "rootEntity": "PurchaseOrder",
  "page": 2, "pageSize": 25, "includeCount": true,
  "select": ["Id", "PONumber", "Status", "TotalAmount"],
  "filters": [
    { "column": "Status", "op": "eq", "value": "Approved" },
    { "column": "TotalAmount", "op": "gte", "value": 5000 }
  ],
  "search": { "term": "PO-2026", "columns": ["PONumber"] },
  "sort": { "column": "OrderDate", "direction": "desc" },
  "nodes": [ { "nodeEntity": "PurchaseOrderLine", "parentLink": "PurchaseOrderId" } ]
}
```

JSON

/forge — Create

```
{
  "rootEntity": "PurchaseOrder", "rootId": null, "operation": "create",
  "content": { "PONumber": "PO-2026-001", "VendorId": 42, "Status": "Draft" },
  "nodes": [ {
    "nodeEntity": "PurchaseOrderLine", "parentLink": "PurchaseOrderId",
    "nodeId": null, "content": { "ItemCode": "JAN-001", "Qty": 10 }
  } ]
}
```

JSON

/forge — Update

```
{
  "rootEntity": "PurchaseOrder", "rootId": "55", "operation": "update",
  "content": { "Status": "Approved" },
  "nodes": [{ "nodeEntity": "PurchaseOrderLine", "parentLink": "PurchaseOrderId",
    "nodeId": "101", "content": { "Qty": 15 } }]
}
```

JSON

/forge — Soft Delete

```
{ "rootEntity": "PurchaseOrder", "rootId": "55", "operation": "delete", "content": {} }
// Generates: UPDATE JANATICS.PurchaseOrder SET IsDeleted = 1 WHERE Id = :rootId
```

JSON